StreamScope

Evidence report for the Hadoop, Spark, MongoDB, and Docker pipeline. Each figure is a screenshot from the WSL Ubuntu session on LAPTOP-H1S3QUIE. The writing under the figure says what the screen proves.

Problem. The project asks which genres carry a lot of rating activity and which of those genres are carried by only a few titles. MovieLens 32M is the interest signal. It is a ratings collection through 12 October 2023, not an OTT view log. TMDB subscription providers for India are the availability signal, stored later as JSON in MongoDB.

What was left out. The package-install progress screen does not show a result. The TMDB website screenshot contains the API key and the read token, so it is not printed here. That key should be regenerated, and it must stay in the TMDB_API_KEY environment variable.

1. Hadoop on one laptop

Hadoop 3.3.6 runs pseudo-distributed on WSL2. There is one NameNode and one DataNode, so replication is 1. Spark 4 was not used, because it needs Java 17. This machine uses Java 11.

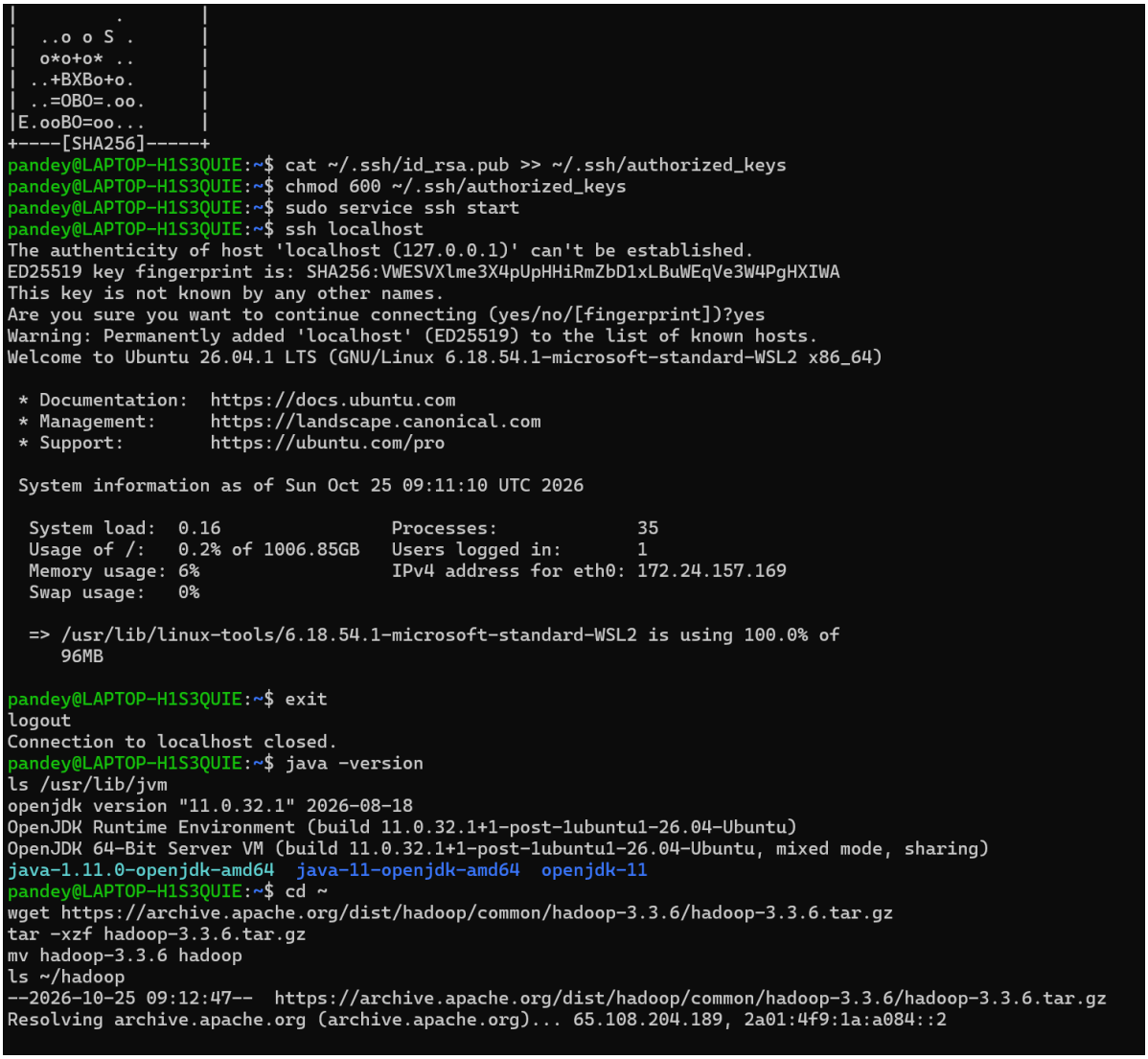


Figure 1. Java 11 and the Hadoop tarball on WSL.

Passwordless SSH to localhost was set up, then java -version showed OpenJDK 11.0.32. Hadoop 3.3.6 was downloaded from archive.apache.org into the home directory. The same terminal later reports the host as LAPTOP-H1S3QUIE and the disk as about 1006 GB, which matches the NameNode capacity.

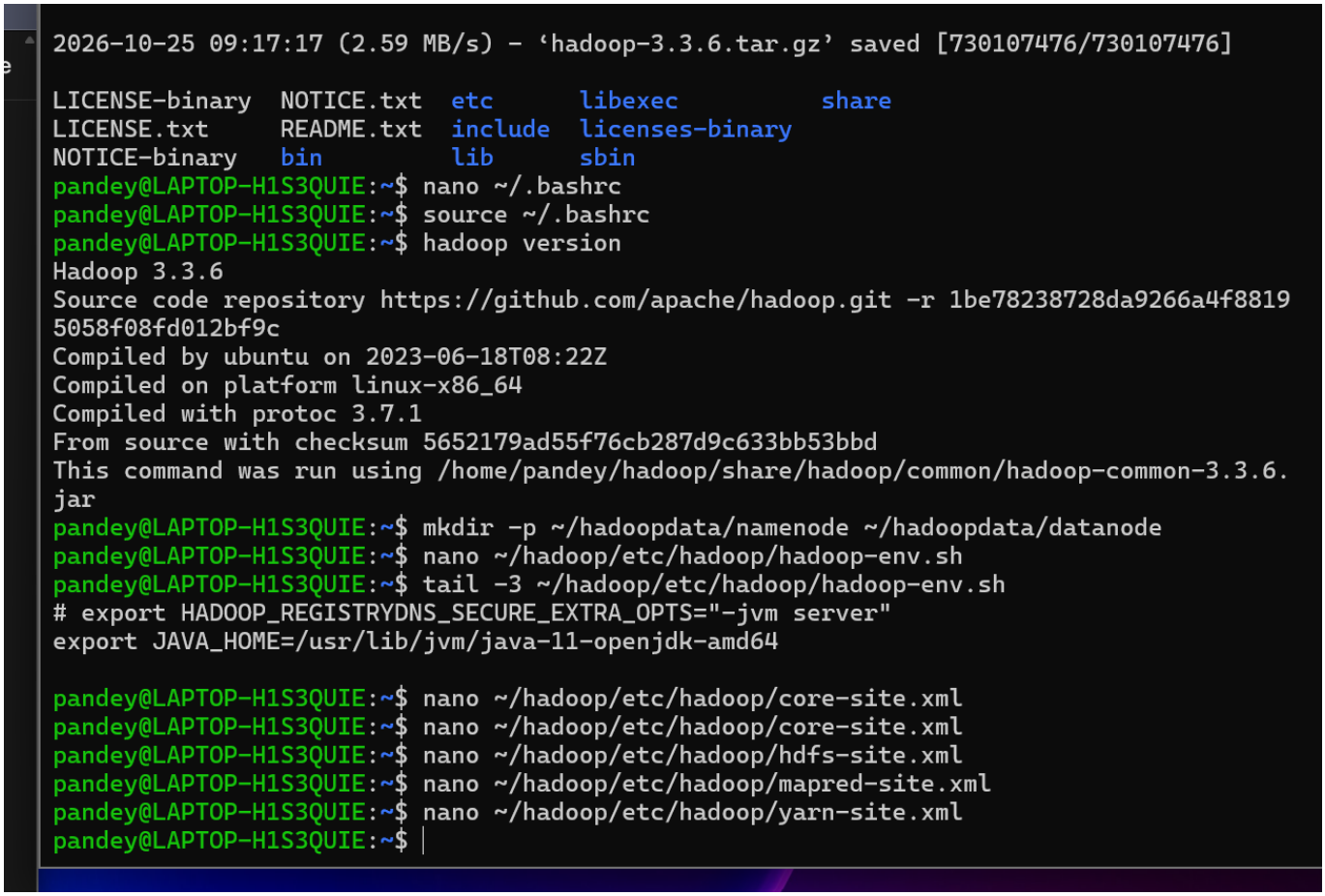


Figure 2. Hadoop 3.3.6 on the path, and the storage directories.

hadoop version prints Hadoop 3.3.6. JAVA_HOME is set to /usr/lib/jvm/java-11-openjdk-amd64 inside hadoop-env.sh. The NameNode and DataNode directories were created at ~/hadoopdata/namenode and ~/hadoopdata/datanode before the site files were edited.

```
jar
pandey@LAPTOP-H1S3QUIE:~$ mkdir -p ~/hadoopdata/namenode ~/hadoopdata/datanode
pandey@LAPTOP-H1S3QUIE:~$ nano ~/hadoop/etc/hadoop/hadoop-env.sh
pandey@LAPTOP-H1S3QUIE:~$ tail -3 ~/hadoop/etc/hadoop/hadoop-env.sh
# export HADOOP_REGISTRYDNS_SECURE_EXTRA_OPTS="-jvm server"
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

pandey@LAPTOP-H1S3QUIE:~$ nano ~/hadoop/etc/hadoop/core-site.xml
pandey@LAPTOP-H1S3QUIE:~$ nano ~/hadoop/etc/hadoop/core-site.xml
pandey@LAPTOP-H1S3QUIE:~$ nano ~/hadoop/etc/hadoop/hdfs-site.xml
pandey@LAPTOP-H1S3QUIE:~$ nano ~/hadoop/etc/hadoop/mapred-site.xml
pandey@LAPTOP-H1S3QUIE:~$ nano ~/hadoop/etc/hadoop/yarn-site.xml
pandey@LAPTOP-H1S3QUIE:~$ cat ~/hadoop/etc/hadoop/core-site.xml | tail -8
cat ~/hadoop/etc/hadoop/hdfs-site.xml | tail -16
grep -c "<configuration>" ~/hadoop/etc/hadoop/*-site.xml
<!-- Put site-specific property overrides in this file. -->
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>


<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/pandey/hadoopdata/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/pandey/hadoopdata/datanode</value>
  </property>
</configuration>

/home/pandey/hadoop/etc/hadoop/core-site.xml:1
/home/pandey/hadoop/etc/hadoop/hdfs-rbf-site.xml:1
/home/pandey/hadoop/etc/hadoop/hdfs-site.xml:1
/home/pandey/hadoop/etc/hadoop/httpfs-site.xml:1
/home/pandey/hadoop/etc/hadoop/kms-site.xml:1
/home/pandey/hadoop/etc/hadoop/mapred-site.xml:1
/home/pandey/hadoop/etc/hadoop/yarn-site.xml:1
pandey@LAPTOP-H1S3QUIE:~$
```

Figure 3. core-site.xml and hdfs-site.xml.

fs.defaultFS is hdfs://localhost:9000. dfs.replication is 1, because this laptop has one DataNode. The NameNode image is file:///home/pandey/hadoopdata/namenode and the DataNode blocks are file:///home/pandey/hadoopdata/datanode. Every *-site.xml under etc/hadoop contains a configuration block, so the files were saved rather than left empty.

2. Dataset loaded and distributed

The three files that matter are ratings.csv, movies.csv, and links.csv. They were placed on HDFS under /streamscope/raw/movielens. fsck and the NameNode UI are the distribution check: the blocks exist, replication is 1, and one DataNode is live.



Figure 4. MovieLens 32M download, checksums, and line count.

ml-32m.zip (238,950,008 bytes) downloaded completely. The zip checksum and the per-file checksums for links.csv, movies.csv, ratings.csv, and tags.csv all printed OK. wc -l ratings.csv printed 32,000,205, which is the header plus 32,000,204 ratings. The certificate warning on files.grouplens.org did not stop the download; the md5 check is what confirms the file.

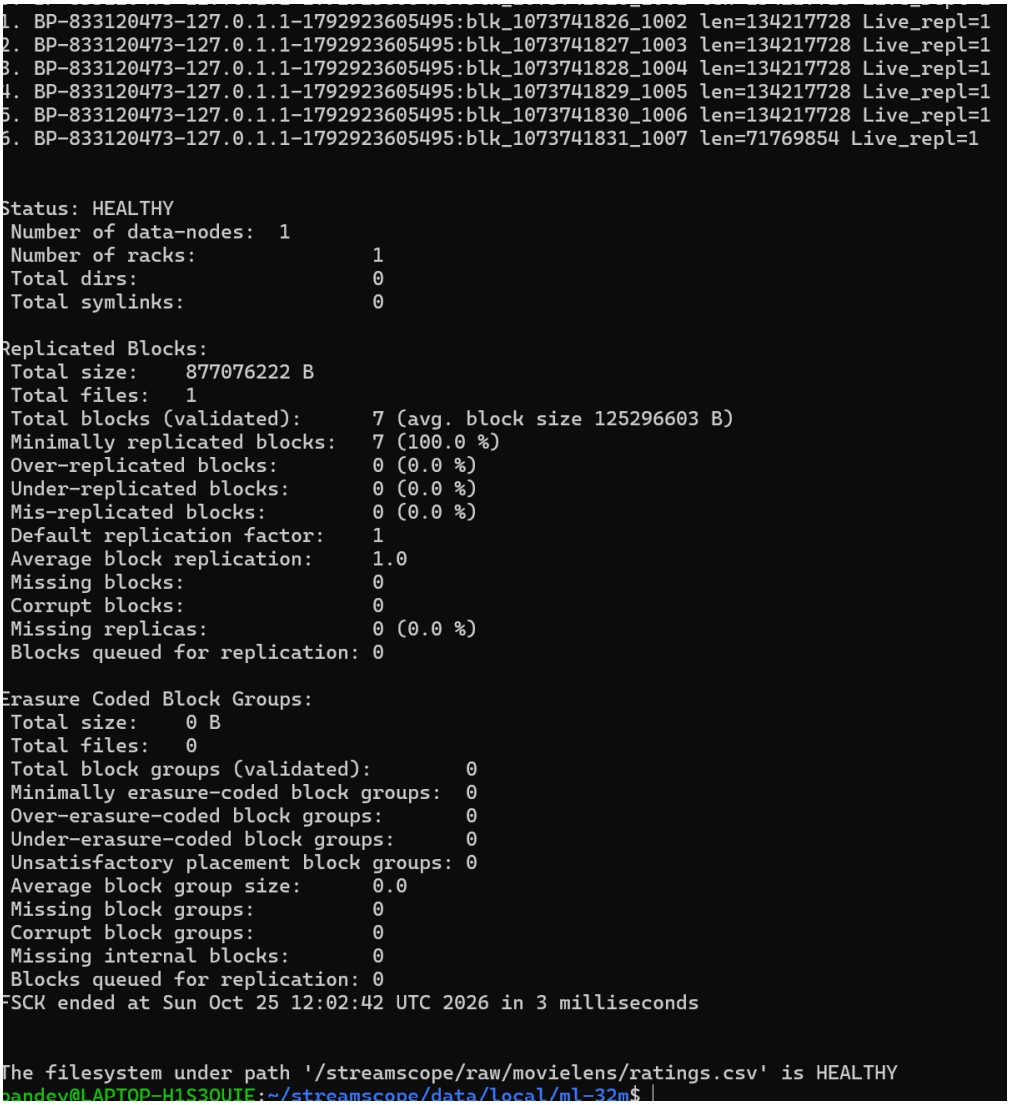


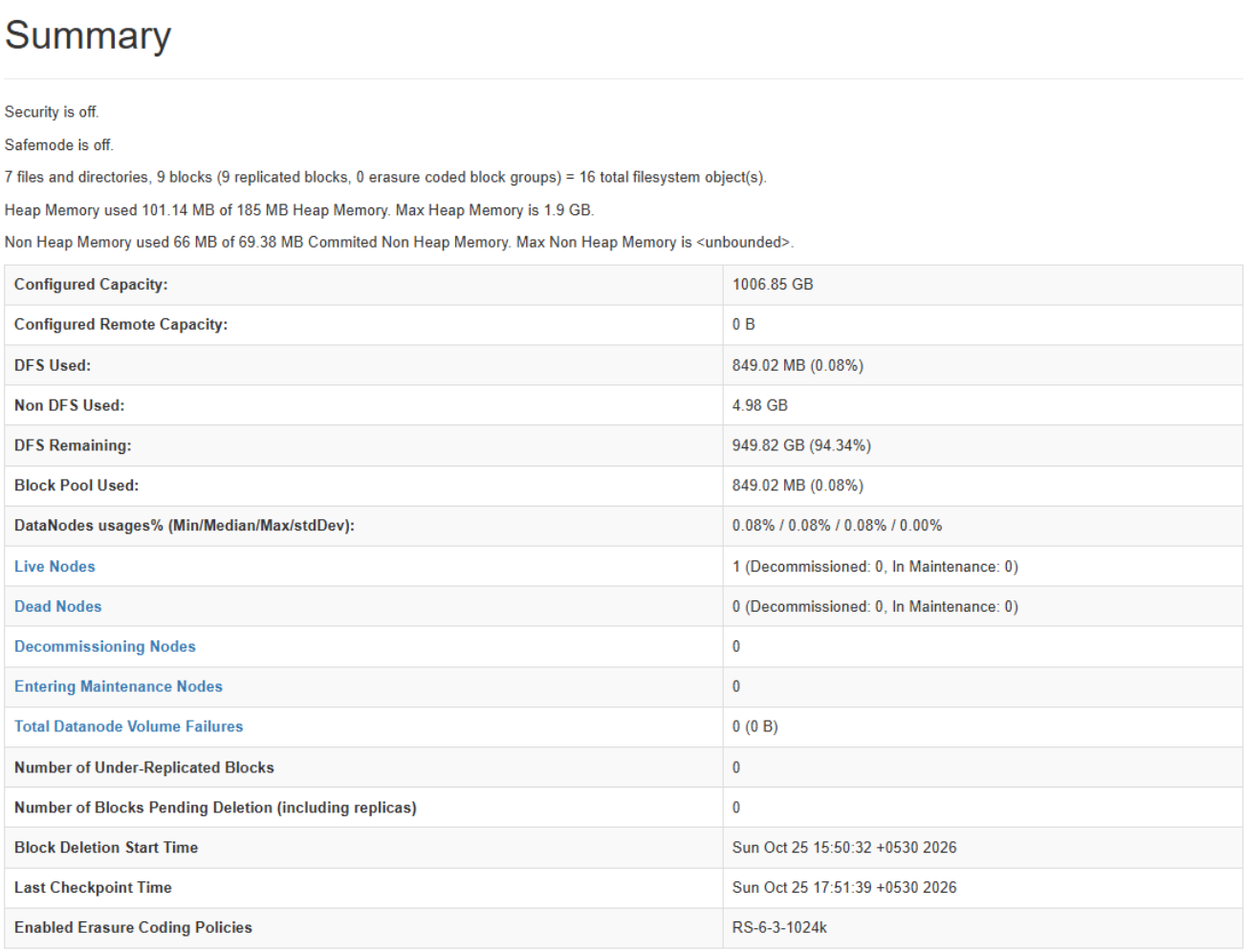
Figure 5. fsck of ratings.csv: HEALTHY, 7 blocks, replication 1.

hdfs fsck on /streamscope/raw/movielens/ratings.csv reports Status HEALTHY, 1 data-node, total size 877,076,222 bytes, and 7 validated blocks. The default replication factor is 1 and there are 0 missing, corrupt, or under-replicated blocks. Six blocks are the full 128 MiB (134,217,728 bytes) and the last block is the remainder, 71,769,854 bytes. Six times 134,217,728 plus 71,769,854 equals 877,076,222, so the file was not truncated. movies.csv and links.csv are smaller than one block, so each occupies one block. That is why the cluster shows 9 blocks while fsck of ratings.csv alone shows 7.



Figure 6. One live DataNode, safe mode off.

The report for the ratings path shows Live datanodes (1): 127.0.0.1:9866 on LAPTOP-H1S3QUIE. DFS used is 848.96 MB, capacity is 1006.85 GB, and about 949.82 GB remains. Num of Blocks is 9. Decommission status is Normal. Cache used at 100 percent is meaningless here because the configured cache capacity is 0.



Summary

Security is off.

Safemode is off.

7 files and directories, 9 blocks (9 replicated blocks, 0 erasure coded block groups) = 16 total filesystem object(s).

Heap Memory used 101.14 MB of 185 MB Heap Memory. Max Heap Memory is 1.9 GB.

Non Heap Memory used 66 MB of 69.38 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	1006.85 GB
Configured Remote Capacity:	0 B
DFS Used:	849.02 MB (0.08%)
Non DFS Used:	4.98 GB
DFS Remaining:	949.82 GB (94.34%)
Block Pool Used:	849.02 MB (0.08%)
DataNodes usages% (Min/Median/Max/stdDev):	0.08% / 0.08% / 0.08% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Sun Oct 25 15:50:32 +0530 2026
Last Checkpoint Time	Sun Oct 25 17:51:39 +0530 2026
Enabled Erasure Coding Policies	RS-6-3-1024k

Figure 7. NameNode UI summary at http://localhost:9870.

Security is off. Safemode is off. The overview lists 7 files and directories, 9 replicated blocks, and 0 under-replicated blocks. Live nodes is 1 and dead nodes is 0. DFS used is 849.02 MB of a 1006.85 GB disk. The NameNode started on Sun 25 Oct 2026 at 15:50:32 IST. This is the screen that shows the upload is distributed on the running cluster, not only present in the shell.

Block Deletion Start Time	Sun Oct 25 15:50:32 +0530 2026
Last Checkpoint Time	Sun Oct 25 16:51:39 +0530 2026
Enabled Erasure Coding Policies	RS-6-3-1024k

NameNode Journal Status

Current transaction ID: 44

Journal Manager	State
FileJournalManager(root=/home/pandey/hadoopdata/namenode)	EditLogFileOutputStream(/home/pandey/hadoopdata/n

NameNode Storage

Storage Directory	Type	State
/home/pandey/hadoopdata/namenode	IMAGE_AND_EDITS	Active

DFS Storage Types

Storage Type	Configured Capacity	Capacity Used	Capacity Remaining	Block Pool Used	Nodes In Service
DISK	1006.85 GB	848.96 MB (0.08%)	949.82 GB (94.34%)	848.96 MB	1

Figure 8. NameNode storage directory in use.

The journal and the image both use /home/pandey/hadoopdata/namenode, and that storage is Active (IMAGE_AND_EDITS). The DFS storage table repeats one disk, 848.96 MB used, and 1 node in service. The SecondaryNameNode checkpoint is a backup of this image. It is not a second NameNode.

3. Spark reads HDFS and builds the genre table

PySpark runs on the host in local mode, with HADOOP_CONF_DIR pointed at the Hadoop configuration, so it opens hdfs://localhost:9000. The count checks the raw file. The genre job is the table the dashboard later stores in MongoDB.

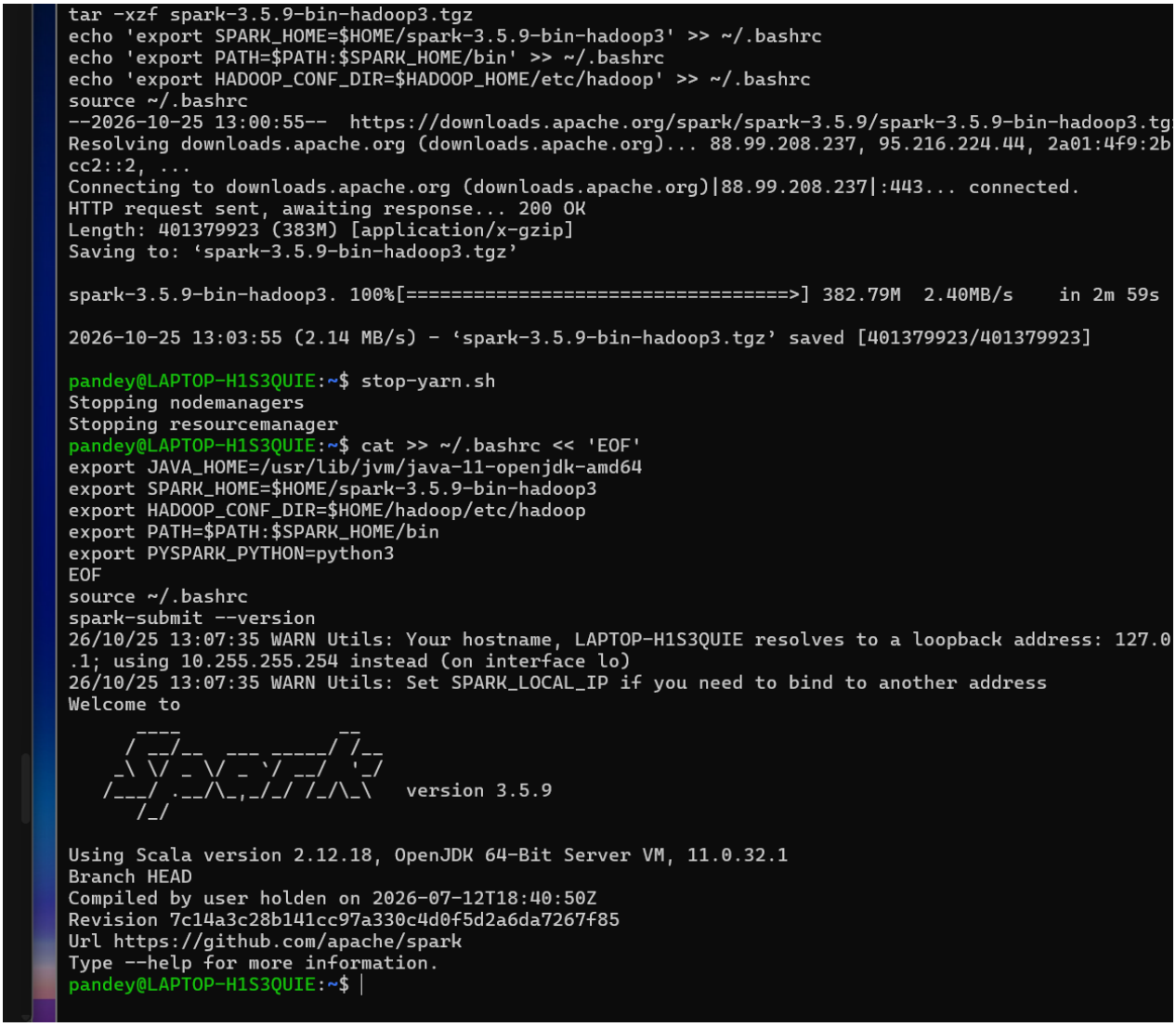


Figure 9. Spark 3.5.9, built for Hadoop 3, on Java 11.

Spark 3.5.9 was unpacked and SPARK_HOME, PATH, and HADOOP_CONF_DIR were added to the shell. spark-submit --version prints Spark 3.5.9, Scala 2.12.18, and OpenJDK 11.0.32. YARN was stopped with stop-yarn.sh before this, so local-mode Spark can use the laptop memory. The hostname warning is only that LAPTOP-H1S3QUIE resolves to 127.0.1.1; Spark then uses another address. It does not mean the install failed.

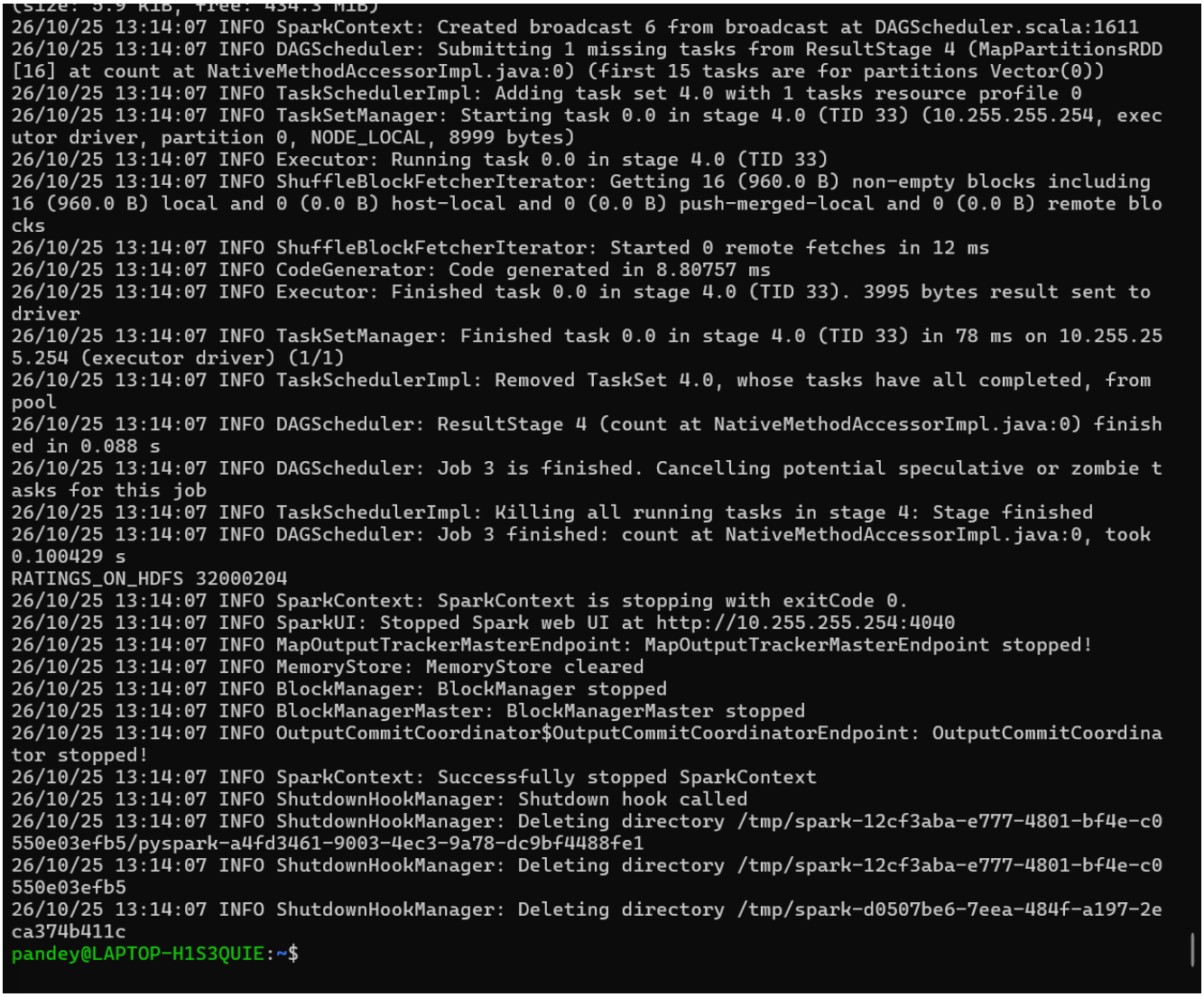


Figure 10. PySpark count of the ratings file on HDFS.

The job read hdfs://localhost:9000/streamscope/raw/movielens/ratings.csv and printed RATINGS_ON_HDFS 32000204, then exited with code 0. That matches the line count after removing the header. This is the proof that Spark, not only the hdfs command, can read the distributed file.

```
asks for this job
26/10/25 13:18:36 INFO TaskSchedulerImpl: Killing all running tasks in stage 10: Stage finished
26/10/25 13:18:36 INFO DAGScheduler: Job 7 finished: showString at NativeMethodAccessorImpl.java:0,
took 0.135951 s
26/10/25 13:18:36 INFO CodeGenerator: Code generated in 18.284667 ms
26/10/25 13:18:36 INFO CodeGenerator: Code generated in 10.406788 ms
+---------------------------------+------------+-----------+-----------+----------+
|genre                            |rating_count|rater_count|movie_count|avg_rating|
+---------------------------------+------------+-----------+-----------+----------+
|Drama                            |13973271    |200914     |33152      |3.6825    |
|Comedy                           |11206925    |200693     |22447      |3.4324    |
|Action                           |9665213     |200208     |9296       |3.4764    |
|Thriller                         |8679464     |200168     |11555      |3.5317    |
|Adventure                        |7590522     |199968     |5156       |3.5234    |
|Sci-Fi                           |5717337     |197981     |4797       |3.4917    |
|Romance                          |5524615     |198689     |10048      |3.545     |
|Crime                            |5373051     |198878     |6704       |3.6918    |
|Fantasy                          |3702759     |193899     |3784       |3.5122    |
|Children                         |2731841     |182409     |4447       |3.4392    |
|Mystery                          |2615322     |188889     |3894       |3.6731    |
|Horror                           |2492315     |174469     |8468       |3.3072    |
|Animation                        |2214562     |173853     |4586       |3.6153    |
|War                              |1594110     |181626     |2225       |3.7917    |
|IMAX                             |1494179     |151832     |195        |3.5933    |
|Musical                          |1159516     |152251     |1033       |3.5543    |
|Western                          |596654      |134707     |1489       |3.6002    |
|Documentary                      |427353      |74356      |9103       |3.6912    |
|Film-Noir                        |304710      |83671      |350        |3.9158    |
| We're Comin' To Get Ya!"" (2014)"|1           |1          |1          |2.0       |
+---------------------------------+------------+-----------+-----------+----------+

26/10/25 13:18:36 INFO FileSourceStrategy: Pushed Filters: IsNotNull(movieId)
26/10/25 13:18:36 INFO FileSourceStrategy: Post-Scan Filters: isnotnull(movieId#18)
26/10/25 13:18:36 INFO FileSourceStrategy: Pushed Filters: IsNotNull(movieId)
26/10/25 13:18:36 INFO FileSourceStrategy: Post-Scan Filters: isnotnull(movieId#42)
26/10/25 13:18:36 INFO MemoryStore: Block broadcast_15 stored as values in memory (estimated size 19
9.6 KiB, free 421.7 MiB)
26/10/25 13:18:36 INFO MemoryStore: Block broadcast_15_piece0 stored as bytes in memory (estimated s
ize 34.8 KiB, free 421.7 MiB)
26/10/25 13:18:36 INFO BlockManagerInfo: Added broadcast_15_piece0 in memory on 10.255.255.254:41307
```

Figure 11. Genre aggregate computed by PySpark.

The job joined ratings to movies, split the genre field on |, and aggregated rating_count, rater_count, movie_count, and avg_rating. Drama is first, with 13,973,271 ratings, 200,914 raters, 33,152 movies, and average 3.6825. The last row, a title fragment from a broken quote in movies.csv, is not a genre. It has 1 rating. The dashboard allow-list drops it and keeps the 19 real genres. A title with two genres is counted in both, so the genre column sums to more than 32,000,204.

4. Docker stores the query side in MongoDB

Hadoop stays on the host. Docker runs MongoDB. The dashboard container uploads the 19-genre table into streamscope.genre_interest and queries it. The 32,000,204 ratings are not copied into MongoDB. Provider documents, when the fetch is finished, go into streamscope.tmdb_providers.

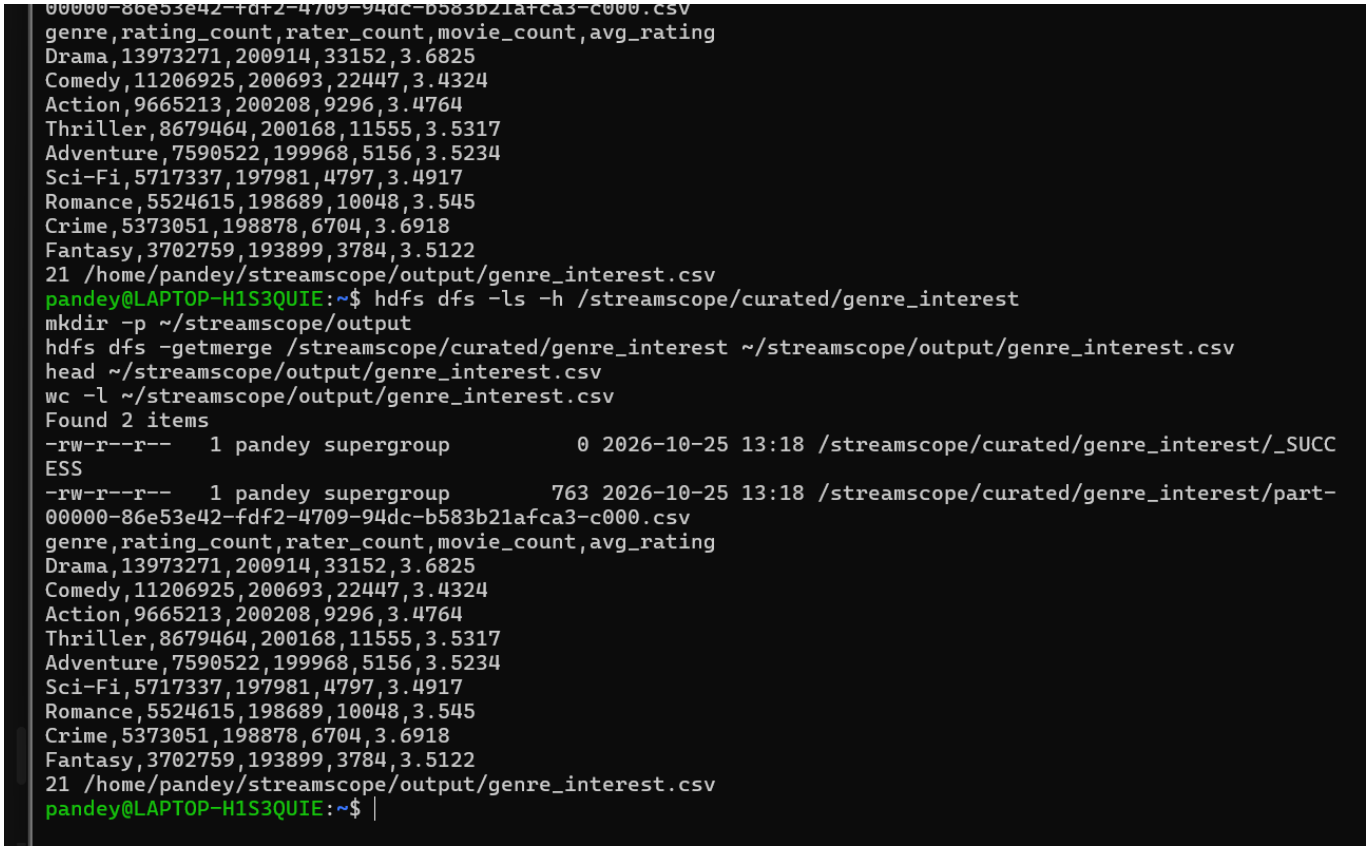


Figure 12. The curated genre table copied out of HDFS.

hdfs dfs -ls shows the Spark output directory: a _SUCCESS file and one part CSV. hdfs dfs -getmerge wrote ~/streamscope/output/genre_interest.csv, and wc -l printed 21, which is the header, 19 genres, and the one bad row. The Docker dashboard mounts this file and uploads the 19 genres into MongoDB. The container does not open hdfs://localhost:9000, because inside a container localhost is the container.

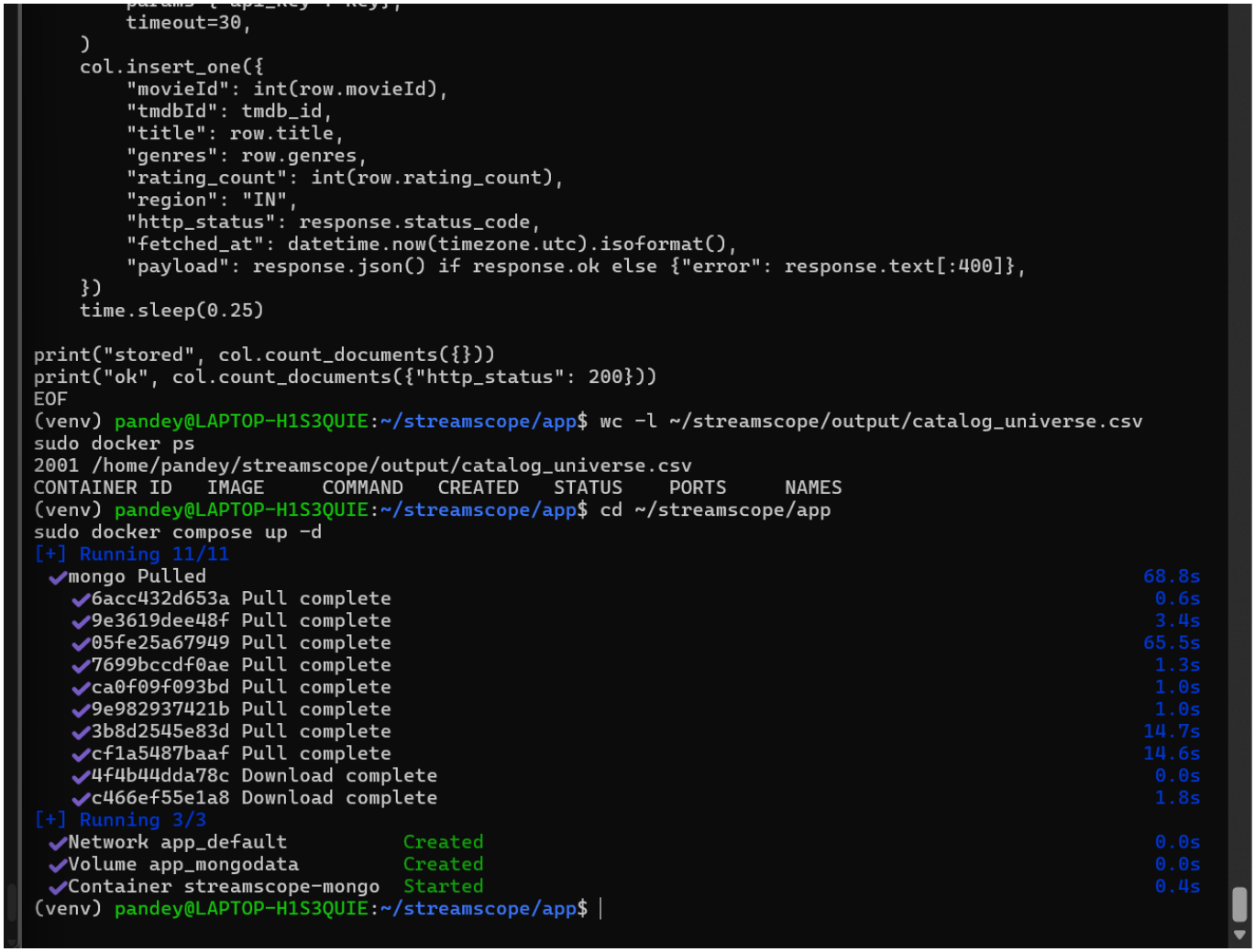


Figure 13. MongoDB started with Docker Compose.

catalog_universe.csv has 2,001 lines: the header plus the 2,000 MovieLens titles Spark kept for the provider study. docker compose up -d pulled mongo:7, created the volume app_mongodata, and started the container streamscope-mongo. That volume is local Docker storage on the laptop, not a cloud database. Provider JSON is inserted into database streamscope, collection tmdb_providers, and only the short API key from the environment is used. The key is not written into the image or this report.

What these screenshots do not yet show

They do not show a finished count of TMDB documents, and they do not show the Streamlit tabs. After fetch_tmdb.py prints a stored count, add one screenshot of a tmdb_providers document and one screenshot of http://localhost:8502. Until that count is 2000, describe the provider collection as incomplete. Do not quote a coverage percentage.